Ex no :	Online Shopping Assistant
Date :	

AIM:

To design and develop an AI-powered online shopping assistant chatbot using Python (Flask) that helps users discover, compare, and select products based on their preferences through an interactive conversational interface.

DESCRIPTION:

In today's fast-growing e-commerce landscape, customers are often overwhelmed by the vast number of products available online. Finding the right product, comparing alternatives, and making informed purchasing decisions can be time-consuming and confusing. The Online Shopping Assistant Chatbot is an intelligent virtual assistant designed to simplify and enhance the online shopping experience.

This project focuses on developing a dynamic chatbot that goes beyond basic search functionality. Instead of manually browsing through multiple pages, users can interact with the chatbot using natural language queries such as "Suggest me a phone under ₹20,000" or "Compare laptops with good battery life." The chatbot interprets user intent and provides personalized product recommendations and comparisons in real time.

The system uses a structured product dataset containing attributes like product name, category, price, rating, and features. By applying rule-based or keyword-based Natural Language Processing (NLP), the chatbot identifies user requirements and filters relevant products accordingly. It can also handle follow-up queries, enabling a smooth conversational shopping experience similar to interacting with a human sales assistant.

The chatbot is integrated into a web-based interface, functioning as an interactive chat widget for e-commerce platforms. The frontend is built using HTML, CSS, and JavaScript to provide a responsive and user-friendly chat interface, while the backend is powered by Python Flask to manage request handling, business logic, and product filtering.

Project Structure:

- **app.py:** Flask backend handling chatbot logic, intent detection, and product recommendation/comparison.
- **templates/index.html:** Structure of the chatbot user interface.
- **static/style.css:** Styling for a clean, modern shopping chat UI.
- **static/script.js:** Handles user input, API calls, and dynamic message rendering.
- **products.json:** Sample dataset containing product details used for recommendations.

Impact:

Enhanced Shopping Experience:
Provides personalized product suggestions, reducing the time and effort required to find suitable items.

Smart Decision Making:
Allows users to compare products based on price, features, and ratings, helping them make informed choices.

24/7 Customer Assistance:
Offers continuous support without the need for human intervention, improving customer satisfaction.

Scalable Solution:
Can handle multiple users simultaneously, making it suitable for large-scale e-commerce platforms.

Reduced Bounce Rate:
Engaging chatbot interaction keeps users on the platform longer, increasing the likelihood of purchase.

PROGRAM CODE:

1) Backend Code :

```
"""
E-commerce Chatbot Backend
Flask application that handles chatbot requests and processes user queries
"""
from flask import Flask, render_template, request, jsonify
import json
import re
from datetime import datetime
app = Flask(__name__)
# Load product dataset
def load_products():
    """Load products from JSON file"""
    with open('products.json', 'r') as f:
    return json.load(f)['products']
    # Global product list
    PRODUCTS = load_products()
    class ChatbotEngine:
    """Rule-based chatbot engine for product recommendations and comparisons"""
```

```
def __init__(self, products):
self.products = products
self.conversation_history = []
def normalize_text(self, text):
"""Normalize user input for better matching"""
return text.lower().strip()
def extract_budget(self, text):
"""Extract budget range from user input"""
# Pattern: "under $100", "$50-100", "budget $200", etc.
patterns = [
r'under\s*\$?\s*(\d+)',
r'max\s*\$?\s*(\d+)',
r'Budget\s*\$?\s*(\d+)',
r'\$(\d+)\s*-\s*\$?(\d+)',
r'(\d+)\s*-\s*(\d+)',
]
for pattern in patterns:
match = re.search(pattern, text, re.IGNORECASE)
if match:
if len(match.groups()) == 1:
return (0, int(match.group(1)))
else:
return (int(match.group(1)), int(match.group(2)))
return None
def extract_category(self, text):
"""Extract product category from user input"""
categories = {
'electronics': ['electronics', 'phone', 'headphones', 'keyboard', 'mouse', 'webcam', 'charger'],
'accessories': ['accessories', 'case', 'protector', 'stand', 'cable', 'organizer', 'hub'],
'storage': ['storage', 'ssd', 'drive'],
'home office': ['home office', 'desk', 'lamp', 'monitor']
}
normalized = self.normalize_text(text)
for category, keywords in categories.items():
        for keyword in keywords:
```

```
            if keyword in normalized:
            return category
        return None
        def extract_brand(self, text):
        """Extra-ct brand from user input"""
        brands = [product['brand'] for product in self.products]
        normalized = self.normalize_text(text)
        for brand in set(brands):
            if self.normalize_text(brand) in normalized:
            return brand
        return None
    def detect_intent(self, text):
        """Detect user intent from input"""
        normalized = self.normalize_text(text)
        # Keywords for different intents
        search_keywords = ['find', 'search', 'looking for', 'want', 'need', 'show me']
        recommend_keywords = ['recommend', 'suggest', 'good', 'what should']
        compare_keywords = ['compare', 'difference', 'vs', 'versus', 'which is better']
        price_keywords = ['price', 'cost', 'cheap', 'expensive', 'budget', 'discount', 'under']
        rating_keywords = ['rating', 'review', 'highly rated', 'popular', 'best rated', 'top rated', 'highest rated']
        if any(keyword in normalized for keyword in compare_keywords):
            return 'compare'
        elif any(keyword in normalized for keyword in price_keywords):
            return 'price_search'
        elif any(keyword in normalized for keyword in rating_keywords):
            return 'rating_search'
        elif any(keyword in normalized for keyword in recommend_keywords):
            return 'recommend'
        else:
            return 'search'
    def search_products(self, query, budget=None, category=None, brand=None, min_rating=None):
        """Search products based on criteria"""
        results = self.products.copy()
        # Filter by category
        if category:
```

```
            results = [p for p in results if self.normalize_text(p['category']) == self.normalize_text(category)]
            # Filter by brand
        if brand:
            results = [p for p in results if self.normalize_text(p['brand']) == self.normalize_text(brand)]
        # Filter by budget
        if budget:
            min_price, max_price = budget
            results = [p for p in results if min_price <= p['price'] <= max_price]
        # Filter by minimum rating
        if min_rating:
            results = [p for p in results if p['rating'] >= min_rating]
        # Filter by keyword in name or brand
        if query and query.lower() != 'all':
            query_normalized = self.normalize_text(query)
        results = [
        p for p in results
        if query_normalized in self.normalize_text(p['product_name']) or
            query_normalized in self.normalize_text(p['brand'])
        ]
        return results
    def format_product(self, product):
        """Format product information for display"""
        features_str = ', '.join(product['features'][:2]) # Show first 2 features
    return f"{product['product_name']} ($${product['price']}) - ⭐ {product['rating']}/5"

    def format_product_details(self, product):
        """Format detailed product information"""
        features_str = ', '.join(product['features'])
        return {
        'name': product['product_name'],
        'brand': product['brand'],
        'price': f"$${product['price']}",
        'rating': f"{product['rating']}/5",
        'category': product['category'],
        'features': features_str
```

```
}
    def generate_response(self, user_input):
    """Generate chatbot response based on user input"""
    # Detect intent and extract parameters
    intent = self.detect_intent(user_input)
    budget = self.extract_budget(user_input)
    category = self.extract_category(user_input)
    brand = self.extract_brand(user_input)
    # Determine what keywords to use for product search
    # If category or brand is found but no product keywords, use 'all'
    # Otherwise use a simplified query
    query = 'all'
    if not category and not brand:
    query = user_input
    # Handle different intents
    if intent == 'compare':
    # Extract products to compare
    products = self.search_products(query, budget=budget, category=category, brand=brand)
    if len(products) >= 2:
        response = "📊 **Product Comparison:**\n\n"
        for i, product in enumerate(products[:3], 1):
        details = self.format_product_details(product)
        response += f"{i}. **{details['name']}**\n Brand: {details['brand']}\n Price: {details['price']}\n Rating:
        {details['rating']}\n Features: {details['features']}\n\n"
    return response.strip()
    else:
        return "❌ I need at least 2 products to compare. Could you provide more details?"
        elif intent == 'price_search':
        if budget:
            min_price, max_price = budget
            products = self.search_products('all', budget=budget, category=category)
        if products:
            response = f"💰 Finding products in your budget (${min_price}-${max_price}):\n\n"
        for product in products[:5]:
            response += f"• {self.format_product(product)}\n"
```

```
return response.strip()
else:
return f"❌ Sorry, I couldn't find products in the ${min_price}-${max_price} range."

else:
    return "💰 Please specify a budget range (e.g., 'under $50' or '$20-100')"
    elif intent == 'rating_search':
        # For rating search, get all products in category (or all products) and sort by rating
        products = sorted(self.search_products('all', category=category),
        key=lambda x: x['rating'], reverse=True)
        if products:
            response = "⭐ **Highest Rated Products:**\n\n"
            for product in products[:5]:
            response += f"• {self.format_product(product)}\n"
            return response.strip()
        else:
            return "❌ No highly rated products found in that category."
            elif intent == 'recommend':
            products = self.search_products(query, budget=budget, category=category,
            brand=brand)
            if products:
                if budget:
                    min_p, max_p = budget
                        response = f"✅ **My Recommendations (${min_p}-
                        ${max_p}):**\n\n"
            else:
                response = "✅ **My Recommendations:**\n\n"
                for product in products[:4]:
                    response += f"• {self.format_product(product)}\n"
                    return response.strip()
else:
    return "❌ I couldn't find matching recommendations. Try different criteria!"
    products = self.search_products(query, budget=budget, category=category, brand=brand)

if products:
    response = "🔍 **Found Products:**\n\n"
```

```
                for product in products[:5]:
                response += f"• {self.format_product(product)}\n"
        if len(products) > 5:
                response += f"\n... and {len(products) - 5} more products"
        return response.strip()
        else:
                suggestions = [
                "Try searching with a category (Electronics, Accessories, Storage, Home Office)",
                "Set a budget range (e.g., 'under $50')",
                "Search by brand name",
                "Ask for recommendations"
    ]
                return f"❌ No products found. Here are some tips:\n" + "\n".join(f"• {s}" for s in suggestions)
# Initialize chatbot engine
                chatbot = ChatbotEngine(PRODUCTS)
@app.route('/')

def index():
    """Render the main chat interface"""
    return render_template('index.html')

@app.route('/chat', methods=['POST'])
def chat():
"""
Handle chat requests
Expected JSON format: {"message": "user message"}
Returns: {"response": "bot response", "timestamp": "timestamp"}
"""
    try:
        data = request.get_json()
        user_message = data.get('message', '').strip()
# Validate input
    if not user_message:
        return jsonify({
        'response': '❌ Please enter a message!',
```

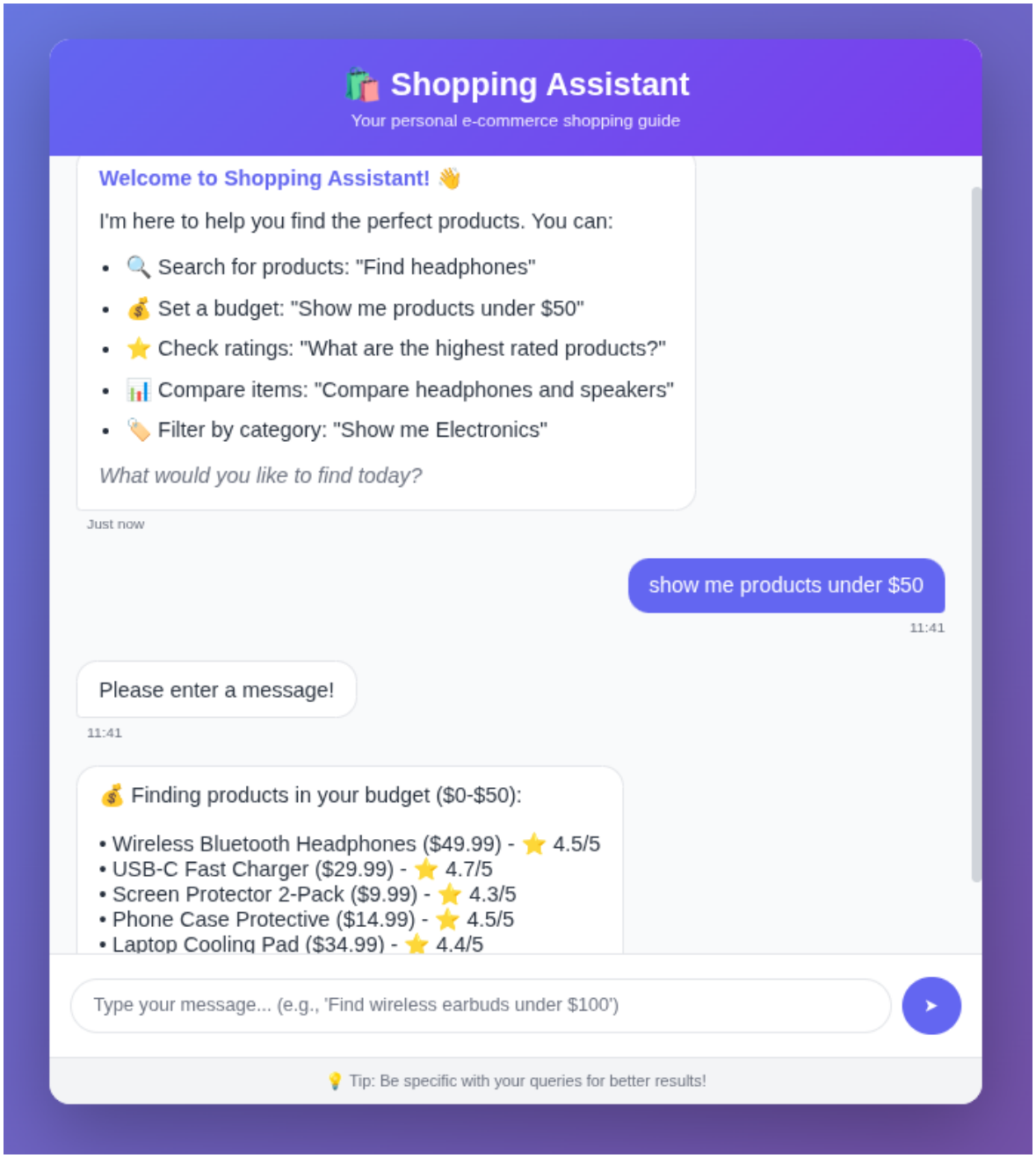
```
'            timestamp': datetime.now().isoformat()
            }), 400
            # Generate bot response
            bot_response = chatbot.generate_response(user_message)
            return jsonify({
            'response': bot_response,
            'timestamp': datetime.now().isoformat()
            }), 200
except Exception as e:
            # Error handling
            print(f"Error: {str(e)}")
            return jsonify({
            'response': f'❌ An error occurred: {str(e)}',
            'timestamp': datetime.now().isoformat()
}), 500

@app.route('/products', methods=['GET'])
def get_products():
            """Get all products (optional endpoint for debugging)"""
            return jsonify({'products': PRODUCTS}), 200

@app.route('/products/search', methods=['GET'])
def search_products_api():
"""Search products via API (optional endpoint)"""
            query = request.args.get('q', '')
            category = request.args.get('category', '')
            budget_max = request.args.get('budget_max', None)
            budget = None
            if budget_max:
            try:
                        budget = (0, float(budget_max))
            except ValueError:
                        pass
                        results = chatbot.search_products(query, budget=budget, category=category if category else None)
                        return jsonify({'results': results, 'count': len(results)}), 200
```

```
if __name__ == '__main__':
# Run Flask server in debug mode
app.run(debug=True, host='127.0.0.1', port=5000)
```

2) Front End (index.html)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>E-commerce Shopping Assistant Chatbot</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body>
    <div class="container">
    <!-- Header -->
    <div class="chat-header">
        <div class="header-content">
            <h1>🛍️ Shopping Assistant</h1>
            <p class="subtitle">Your personal e-commerce shopping guide</p>
        </div>
    </div>
<!-- Chat Window -->
    <div class="chat-window" id="chatWindow">
<!-- Welcome Message -->
        <div class="message bot-message">
            <div class="message-content">
                <p><strong>Welcome to Shopping Assistant! 👋</strong></p>
                <p>I'm here to help you find the perfect products. You can:</p>
                <ul style="margin: 10px 0; padding-left: 20px;">
                    <li>🔍 Search for products: "Find headphones"</li>
                    <li>💰 Set a budget: "Show me products under $50"</li>
                    <li>⭐ Check ratings: "What are the highest rated products?"</li>
                    <li>📊 Compare items: "Compare headphones and speakers"</li>
```

```
                                        <li>🏷️ Filter by category: "Show me Electronics"</li>
                                    </ul>
                                <p style="margin-top: 10px;"><em>What would you like to find today?</em></p>
                            </div>
                            <div class="message-time">Just now</div>
                            </div>
                        </div>
<!-- Input Area -->
<div class="input-container">
<form id="chatForm" onsubmit="sendMessage(event)">
<input
                        type="text"
                        id="userInput"
                        class="message-input"
                        placeholder="Type your message... (e.g., 'Find wireless earbuds under $100')"
                        autocomplete="off"
                        >
                        <button type="submit" class="send-button" id="sendBtn">
                        <span class="send-icon">➤</span>
                        </button>
</form>
                        </div>
<!-- Info Footer -->
                        <div class="footer">
                        <p>💡 Tip: Be specific with your queries for better results!</p>
                        </div>
</div>
<!-- Typing Indicator Template (hidden by default) -->
<template id="typingIndicator">
<div class="message bot-message">
                        <div class="message-content">
                            <div class="typing-indicator">
                                <span></span>
                                <span></span>
                                <span></span>
```

```
                </div>
            </div>
</div>
</template>
<script src="{{ url_for('static', filename='script.js') }}"></script>
</body>
</html>
```

Output :